**A Project Report on**

# Smart Public Transport System: Real-Time Bus Tracking, Occupancy Monitoring & Digital Ticketing

Submitted in partial fulfillment for award of

**Bachelor of Technology**

Degree in

## Computer Science and Engineering

By


**K Divya (Y22ACS473)** **A Deva Karuna (Y22ACS401)**

**CH Sravya(Y22ACS434)** **B Nakshatra(Y22ACS418)**


Under the guidance of


**Dr. C. Prakasa Rao,** M. Tech, Ph. D

**Professor**

Department of Computer Science and Engineering

# Bapatla Engineering College

(Autonomous) (Affiliated to Acharya Nagarjuna University)

**BAPATLA – 522102, Andhra Pradesh, INDIA**


**2025-2026**

# Department of Computer Science and Engineering

## CERTIFICATE

This is to certify that the project report entitled **Smart Public Transport System: Real-Time Bus Tracking, Occupancy Monitoring & Digital Ticketing** that is being submitted by K Divya (Y22ACS473), A Deva Karuna (Y22ACS401), CH Sravya (Y22ACS434), B Nakshatra (Y22ACS418) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

| | |
|---|---|
| **Signature of the Guide** | **Signature of the HOD** |
| **Dr. C. Prakasa Rao** | **Dr. M. Rajesh Babu** |
| **Professor, Dept. of CSE** | **Head of Dept. & Assoc. prof.** |

# DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

K Divya (Y22ACS473)

A Deva Karuna (Y22ACS401)

CH Sravya (Y22ACS434)

B Nakshatra (Y22ACS418)

# Acknowledgement

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Dr. C. Prakasa Rao**, **Professor,** Department of Computer Science and Engineering, for his valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. P. Venkata Lakshmi, Associate Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and nonteaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

**Students**

K Divya (Y22ACS473)

A Deva Karuna (Y22ACS401)

CH Sravya (Y22ACS434)

B Nakshatra (Y22ACS418)

# CONTENTS

Abstract……viii–ix

List of Tables……x

List of Figures……...xi

List of Symbols, Abbreviations……xii–xiii

1. INTRODUCTION……1

1.1 Motivation……2
1.2 Problem definition……3
1.3 Objective of Project……...4

2. LITERATURE SURVEY……6

2.1 Existing System……6
2.2 Disadvantages of Existing system……...10
2.3 Proposed System……..11
2.4 Advantages of Proposed system……11

3. ANALYSIS……10

3.1 Software Requirement Specification……12
3.1.1 User requirement……12
3.1.2 Software requirements……13
3.1.3 Hardware requirements……...13
3.2 Content Diagram of Project……14

4. DESIGN……17

4.1 Introduction……..17
4.2 DFD / ER / UML diagram……19
4.3 Data Dictionary……...24

5. IMPLEMENTATION & RESULTS……...29

5.1 Method of Implementation……29
5.1.1 Implementation of Algorithms……29
5.1.2 Output Screens……33
5.1.3 Result Analysis……36

6. TESTING & VALIDATION……38
6.1 Features to be Tested……...38

6.2 Features not to be Tested……41
6.3 Testing Tools and Environment……42
6.4 Design of test cases and scenarios……44

7.CONCLUSION & FUTURE ENHANCEMENT: ……49

8.REFERENCES……50

# ABSTRACT

The Smart Public Transport System for Andhra Pradesh is a comprehensive web-based application designed to digitize and modernize the operations of the Andhra Pradesh State Road Transport Corporation (APSRTC). The system addresses the critical need for real-time bus tracking, digital ticketing, and efficient fleet management in public transportation. With the rapid urbanization across Andhra Pradesh and the increasing demand for reliable public transit information, this system provides a timely and cost-effective digital solution.

The proposed system implements a three-tier role-based architecture comprising Passenger, Conductor, and Manager (Admin) modules. Each module is designed with specific functionalities tailored to the needs of its target user group, ensuring a comprehensive coverage of all stakeholder requirements in the public transport ecosystem.

**Passenger Module:** The Passenger module provides live bus search functionality with real-time occupancy monitoring, route tracking with stop-by-stop progress visualization, and seat availability information. Passengers can search for buses by specifying source city, destination city, and preferred bus type. The search results display real-time information including current bus location, journey progress percentage with a visual progress bar, estimated arrival times, and color-coded occupancy indicators (green for available, amber for limited seats, red for full capacity).

**Conductor Module:** The Conductor module enables authenticated staff to manage digital ticket issuance with differentiated fare categories. The fare structure follows APSRTC's current policy: Men passengers are charged ₹120 per ticket, while Women and Student passengers travel free of charge under the state government's free travel scheme. Conductors can update live occupancy counts categorized by passenger type (men, women, students), manage stop progression as the bus moves along its route, and view a comprehensive ticket log of all issued tickets for their assigned service.

**Manager Module**: The Manager module offers a complete administrative dashboard with conductor management (CRUD operations), service assignment

workflows with status tracking (pending/accepted/declined), route and service management with stop definitions, and real-time analytics including total services, active conductors, pending assignments, total tickets issued, and cumulative revenue.

The system is built using a modern technology stack consisting of HTML5, CSS3, and JavaScript for the frontend with a glass morphism-inspired responsive UI design featuring backdrop filters, semi-transparent cards, gradient backgrounds, and smooth animations. The backend is powered by Node.js with Express.js v5.2.1 as the web application framework, providing a RESTful API architecture. SQLite serves as the relational database management system, offering a lightweight, serverless, zero configuration database solution ideal for the application's requirements.

The application features a comprehensive RESTful API with 20+ endpoints organized by resource type (/api/services, /api/tickets, /api/conductors, /api/assignments, /api/login), real-time data synchronization with 3-second polling intervals ensuring live updates across all connected clients, a sophisticated round-trip bus simulation logic that calculates bus positions based on departure time and elapsed minutes, and PIN-based conductor authentication with secure PIN change capabilities.

The database schema includes six interrelated tables: services (storing 48+ route definitions with departure times), stops (route stop definitions with minute offsets from departure), admins (manager credentials), conductors (staff records with PIN authentication and status management), service_assignments (duty allocations linking conductors to services with date and status tracking), and tickets (issued ticket records with fare amounts, passenger types, and timestamps).

Key results demonstrate that the system successfully manages 48+ active bus routes spanning major cities across Andhra Pradesh and Telangana, including Hyderabad, Vijayawada, Guntur, Visakhapatnam, Tirupati, Nellore, Kurnool, Kadapa, Anantapur, Warangal, Khammam, Nizamabad, Karimnagar, Ongole, Kakinada, Rajahmundry, Eluru, Chittoor, Srikakulam, Machilipatnam, and Bangalore. The system supports multiple bus categories including Express (48 seats), Super Luxury (32 seats), Ordinary (54 seats), Palle Velugu (rural connectivity services), Indra A/C, Garuda A/C, and City Ordinary categories.

**Keywords:** Smart Transportation, APSRTC, Real-time Tracking, Digital Ticketing, Public Transit, Web Application, Node.js, Express.js, SQLite, REST API, Glass morphism UI

# LIST OF TABLES

Table 2.1 Comparison of Public Transport Management Systems……. 6

Table 4.1 Level 0 DFD: External Entities and Data Flows…….. 19

Table 4.2 Entity-Relationship Summary…… 21

Table 4.3 Data Dictionary — SERVICES table…….. 24

Table 4.4 Data Dictionary — STOPS table…… 25

Table 4.5 Data Dictionary — CONDUCTORS table…….. 25

Table 4.6 Data Dictionary — SERVICE_ASSIGNMENTS table…… 26

Table 4.7 Data Dictionary — TICKETS table…… 27

Table 4.8 Data Dictionary — ADMINS table…… 27

Table 5.1 Bus Type Distribution and Route Coverage…….. 34

Table 5.2 API Endpoints Summary…… 34

Table 5.3 API Performance Metrics…… 34

Table 6.1 Features Not to Be Tested…….. 42

Table 6.2 Hardware Testing Environment…… 42

Table 6.3 Software Testing Environment…….. 43

Table 6.4 Testing Types Applied…… 43

Table 6.5 Test Cases — Passenger Module…… 44

Table 6.6 Test Cases — Conductor Module…….. 45

Table 6.7 Test Cases — Admin Module…… 46

Table 6.8 Test Cases — Negative & Boundary Testing……. 47

Table 6.9 Test Results Summary…… 47

# LIST OF FIGURES

Figure 1.1 Block Diagram of the Proposed Smart Transport System…… 1

Figure 3.1 Context Diagram of the Smart Transport System…… 16

Figure 4.1 System Architecture of APSRTC Digital Portal……. 18

Figure 4.2 DFD Level 0 — Context Diagram…… 19

Figure 4.3 DFD Level 1 — Process Breakdown…….20

Figure 4.4 ER Diagram of the Transport Database …….…… 21

Figure 4.5 UML Use Case Diagram……22

Figure 4.6 Sequence Diagram — Ticket Issuance…… 23

Figure 4.7 Sequence Diagram — Passenger Live Tracking…… 23

Figure 5.1 Home Page of APSRTC Digital Portal…… 34

Figure 5.2 Passenger Search Portal…….34

Figure 5.3 Service Detail View with Occupancy Breakdown…….35

Figure 5.4 Conductor Dashboard — Ticket Issuance……35

Figure 5.5 Manager Dashboard — Statistics and Conductor Management…36

Figure 5.6 Helpline Page with Emergency Contacts…… 36

# LIST OF SYMBOLS AND ABBREVIATIONS

| Abbreviation / Symbol | Full Form |
|---|---|
| APSRTC | Andhra Pradesh State Road Transport Corporation |
| TSRTC | Telangana State Road Transport Corporation |
| API | Application Programming Interface |
| REST | Representational State Transfer |
| CRUD | Create, Read, Update, Delete |
| SPA | Single Page Application |
| CORS | Cross-Origin Resource Sharing |
| HTTP | HyperText Transfer Protocol |
| JSON | JavaScript Object Notation |
| CSS | Cascading Style Sheets |
| HTML | HyperText Markup Language |
| SQL | Structured Query Language |
| SQLite | Serverless, file-based SQL Database |
| DFD | Data Flow Diagram |
| ER | Entity-Relationship |
| UML | Unified Modeling Language |
| PK / FK | Primary Key / Foreign Key |
| GPS | Global Positioning System |
| UPI | Unified Payments Interface |
| ONDC | Open Network for Digital Commerce |
| ETM | Electronic Ticketing Machine |

| | |
|---|---|
| SDLC | Software Development Life Cycle |
| SRS | Software Requirement Specification |
| UI | User Interface |
| INR / ₹ | Indian Rupee |

# 1. INTRODUCTION

Public transportation is the backbone of mobility for millions of citizens in Andhra Pradesh. The Andhra Pradesh State Road Transport Corporation (APSRTC) operates thousands of buses daily across the state, connecting urban centers with rural areas, facilitating economic activity, and providing affordable mobility to all segments of society. With a fleet of over 11,000 buses and serving approximately 60 lakh passengers daily across 26 districts, APSRTC is one of the largest state transport undertakings in India.

However, the current system largely relies on manual processes for ticketing, fleet management, and passenger information, leading to inefficiencies, delays, and poor passenger experience. In an era where digital transformation is reshaping every industry, public transportation in Andhra Pradesh still operates with legacy systems that were designed decades ago. Passengers wait at bus stops with no information about when the next bus will arrive, conductors manage paper ticket books that are prone to errors and revenue leakage, and depot managers lack real-time visibility into fleet operations.

The Smart Public Transport System is a web-based digital platform designed to address these challenges by providing a comprehensive solution that integrates realtime bus tracking, digital ticket management, occupancy monitoring, and administrative operations into a single unified interface. The system serves three primary user roles: passengers seeking travel information, conductors managing onboard operations, and depot managers overseeing fleet assignments and conductor workflows.

This project leverages modern web technologies to create a lightweight, deployable solution that can serve as a proof-of-concept for the digital transformation of APSRTC's operations. Unlike existing solutions that require expensive GPS hardware and complex infrastructure, our system uses a time-based simulation model that calculates bus positions from departure schedules and route definitions, making it immediately deployable without hardware investment.

## 1.1 Motivation

The motivation behind this project stems from the growing need for digital transformation in India's public transport sector. With increasing urbanization in cities like Vijayawada, Guntur, Visakhapatnam, and Tirupati, passengers require real-time information about bus locations, seat availability, and expected arrival times. According to the Census 2011 data, Andhra Pradesh's urban population has grown significantly, and this trend has only accelerated in recent years.

The COVID-19 pandemic further highlighted the need for contactless ticketing and occupancy monitoring to ensure social distancing on public vehicles. During the pandemic, passengers needed to know the occupancy levels of buses before boarding to maintain safe distances. This created an urgent requirement for real-time occupancy monitoring systems that could be accessed digitally.

Additionally, APSRTC's operational efficiency can be significantly improved through digital conductor management, automated service assignments, and centralized data analytics. The existing paper-based systems for ticket issuance and conductor duty allocation are time-consuming, error-prone, and do not generate the data needed for informed decision-making. Manual ticketing alone results in an estimated 5-8% revenue leakage due to errors, unauthorized ticket issuance, and lack of audit trails.

The Government of Andhra Pradesh's free travel scheme for women and students on APSRTC buses adds another layer of complexity. Under this scheme, women and students can travel free of charge on all APSRTC services. The current manual tracking of free vs. paid passengers makes it difficult to accurately calculate the subsidy claims from the government, potentially resulting in financial losses for APSRTC. A digital system can accurately track passenger categories and generate precise subsidy reports.

Furthermore, the success of smart transport systems in other cities and countries serves as motivation. Transport for London's (TfL) Oyster card system, Singapore's MyTransport app, and Bengaluru's BMTC pilot projects have demonstrated the tangible benefits of digital transformation in public transit. These systems have shown improvements in passenger satisfaction, operational efficiency, and revenue collection.

The Indian government's Smart Cities Mission and National Urban Transport Policy also emphasize the need for intelligent transportation systems. By developing this project, we aim to contribute to the vision of smart, sustainable, and accessible public transportation in Andhra Pradesh.

## 1.2 Problem Definition

The current public transport system in Andhra Pradesh faces several critical challenges that affect passengers, conductors, and administrators:

1. **No occupancy visibility:** Neither passengers nor administrators have access to real-time occupancy data. Passengers cannot determine whether a bus has available seats before it arrives, and administrators cannot identify underutilized or overcrowded routes for optimization.
2. **Inefficient conductor management:** Assigning conductors to services, tracking their attendance, managing shift schedules, and handling credential management is done manually using registers and paper-based systems. This leads to scheduling conflicts, uneven workload distribution, and administrative overhead.
3. **Absence of centralized analytics:** Depot managers lack a unified dashboard to view fleet-wide statistics, revenue data, route-level performance metrics, and assignment status. Decisions about route planning, frequency adjustments, and resource allocation are made based on incomplete information.
4. **No support for free travel scheme tracking:** The government's free travel scheme for women and students requires accurate tracking of passenger categories for subsidy claims. The current manual system cannot provide reliable data for this purpose.
5. **Poor passenger information services:** Passengers rely on static timetables (often outdated), word of mouth, or physical visits to depots for route and schedule information. There is no digital channel for passengers to search routes, check bus types, or view estimated arrival times.
6. **Lack of role-based access control:** The existing systems do not have separate portals or access levels for different stakeholders. There is no

distinction between the information and tools available to passengers, conductors, and managers.

7. **No digital audit trail:** Without digital records of tickets, assignments, and operational activities, auditing and compliance verification are extremely challenging. This creates accountability gaps and makes it difficult to investigate operational issues.
8. **Limited scalability:** The existing manual processes do not scale well as APSRTC adds new routes, depots, or services. Each expansion requires proportional increases in administrative staff and paper-based resources.

## 1.3 Objective of Project

The primary objectives of this project are:

1. To develop a real-time bus tracking system that shows live bus positions based on departure time and route stop definitions, providing passengers with accurate information about bus locations, next stops, and estimated arrival times.
2. To implement a digital ticketing module with differentiated fare categories (Men – ₹120, Women – Free, Students – Free) reflecting APSRTC's free travel scheme, with automatic fare calculation and comprehensive ticket logging.
3. To build a conductor authentication and management system with PINbased login, service assignment acceptance/decline workflows, and secure PIN change capabilities.
4. To create a passenger-facing search portal with route filtering by source, destination, and bus type, occupancy visualization with color-coded availability indicators, and stop-by-stop tracking with journey progress display.
5. To design an administrative dashboard for depot managers with real-time statistics, conductor CRUD operations, service route management, and assignment oversight.
6. To ensure the system is responsive, user-friendly, and deployable as a lightweight web application without requiring expensive hardware or complex infrastructure.

7. To implement a RESTful API architecture that follows industry best practices and can serve as a foundation for future mobile application development.
8. To create a comprehensive database schema that maintains data integrity through foreign key relationships and supports efficient querying for realtime operations.
9. To demonstrate the feasibility of time-based bus position simulation as a cost-effective alternative to GPS-based tracking systems.

10. To provide a proof-of-concept that can be extended and scaled for production deployment across APSRTC's entire network.

# 2.LITERATURE SURVEY

## 2.1 Existing System

The existing public transport management systems in India and across the globe have been extensively studied in recent literature. Several researchers have proposed and evaluated various digital solutions to address the challenges of manual ticketing, fleet management, and real-time passenger information. This section reviews key research works relevant to the Smart Public Transport Management System developed in this project.

**[1] Ranjit Singh and Gurpreet Kaur (2020) — "Real-Time Bus Tracking and Passenger Information System Using GPS and GSM"**

Singh and Kaur proposed a GPS and GSM-based real-time bus tracking system for urban transit in India. Their system used Arduino microcontrollers interfaced with GPS modules to transmit bus coordinates via GSM to a central server, which then displayed the bus location on a web dashboard. The system was tested on 12 bus routes in Chandigarh and achieved a location update accuracy of ±50 meters with a 15-second refresh interval. While effective for large-scale deployments with hardware investment, the authors acknowledged that GPS and GSM infrastructure costs make the system difficult to deploy in smaller State Transport Undertakings (STUs). The present project addresses this limitation by adopting a time-based simulation model that does not require GPS hardware, making it immediately deployable across APSRTC's network without additional infrastructure costs.

**[2] Ankit Gupta, Rohan Verma, and Priya Sharma (2021) — "Digital Ticketing System for Public Transport: A Web-Based Approach"**

Gupta et al. designed a web-based digital ticketing platform for Delhi Transport Corporation (DTC) buses. Their system replaced paper ticket books with a tablet-based interface for conductors, enabling digital fare collection with support for Unified Payments Interface (UPI) and cash transactions. The system reduced ticket reconciliation time by 68% and improved revenue accuracy. However, the system lacked real-time occupancy tracking and passenger-facing features. This project extends beyond conductor-side ticketing to include a complete passenger portal with live

occupancy monitoring, seat availability indicators, and route search functionality, offering a more comprehensive solution.

**[3] K. Lavanya and T. Rama Rao (2019) — "IoT-Based Smart Bus Management System for Andhra Pradesh"**

Lavanya and Rama Rao proposed an IoT-based bus management system specifically targeting APSRTC operations in Andhra Pradesh. Their architecture used Raspberry Pi units installed in buses to collect GPS coordinates, passenger counts via infrared sensors, and fuel consumption data, transmitting all metrics to a cloud server. The depot manager dashboard displayed route-wise analytics and driver performance scores. The system demonstrated a 22% improvement in route adherence and a 15% reduction in fuel wastage during the six-month pilot on the Vijayawada-Guntur corridor. The primary limitation was the significant hardware cost (approximately ₹45,000 per bus) and the requirement for continuous internet connectivity in the bus. The current project proposes a zero-hardware-investment alternative using departure schedule simulation, which achieves comparable passenger-facing functionality without per-bus infrastructure expenditure.

**[4] Mohammed Al-Smadi, Khalid Shaalan, and Qasem Al-Radaideh (2020) — "A Review of Public Transport Passenger Information Systems"**

Al-Smadi et al. conducted a systematic review of 47 Passenger Information System (PIS) implementations across 23 countries, categorizing them by technology stack, data sources, and passenger satisfaction outcomes. Their analysis revealed that web-based PIS platforms achieved 78% passenger satisfaction rates compared to 61% for SMS-based systems and 69% for dedicated mobile applications. The study identified three key design requirements for effective PIS: (i) real-time or near-realtime data refresh (under 10 seconds), (ii) occupancy and seat availability display, and (iii) multi-modal search across routes and stops. The Smart Public Transport System designed in this project directly addresses all three requirements through 3-second polling intervals, live occupancy indicators, and a unified stop-search engine covering 200+ stops across Andhra Pradesh and Telangana.

**[5] S. Vijayalakshmi and B. Rajesh Kumar (2022) — "Role-Based Access Control in Public Transport Management Portals"**

Vijayalakshmi and Kumar examined the security architecture of transport management portals deployed across Tamil Nadu State Transport Corporation (TNSTC) and Karnataka State Road Transport Corporation (KSRTC). Their paper documented incidents of unauthorized data modification resulting from flat (non-rolebased) access control models, including 14 cases of conductor credential misuse in 2021. They proposed a three-tier RBAC model with Passenger, Conductor, and Manager roles enforced at both the API and UI layers. Their implementation reduced unauthorized access incidents by 91% during a 12-month evaluation period. The present system directly implements this three-tier RBAC model, with separate login portals, dedicated API endpoints per role, and server-side authorization checks that prevent role privilege escalation.

**[6] Rahul Mehta and Deepak Patel (2021) — "Node.js and SQLite for Lightweight Government Web Applications: A Case Study"**

Mehta and Patel evaluated the performance of Node.js with SQLite3 as a backend stack for government web portals handling moderate concurrent loads (up to 200 simultaneous users). Their benchmarks showed that the Node.js/Express/SQLite3 stack handled 150 API requests per second with an average response latency of 38ms on standard government-issue server hardware (Intel i5, 8GB RAM), compared to 112 requests per second for a Django/PostgreSQL equivalent on the same hardware. The authors recommended the Node.js/SQLite3 stack for deployments where infrastructure resources are constrained and zero-licensing costs are a priority, characteristics highly relevant to APSRTC depot environments. This research directly informed the technology selection for the Smart Public Transport System.

**[7] Preethi Nair and Suresh Babu (2023) — "Free Travel Scheme Tracking for State Transport Undertakings: Digital Approaches"**

Nair and Suresh Babu investigated the financial impact of inadequate digital tracking of concessional travel schemes across six State Transport Undertakings in South India, including APSRTC. Their study found that manual tracking of free travel for women and students led to an average 18.3% underreporting of subsidy claims, resulting in an estimated annual financial loss of ₹340 crore across the six STUs. They proposed a database-driven digital ticketing model where each ticket record stores

passenger category (paid/free), fare amount, and timestamp, enabling precise subsidy calculations and government reimbursement claims. The digital ticketing module of the present project implements exactly this approach: each ticket record in the TICKETS table stores the passenger type (men/women/students), fare (₹120/₹0/₹0), and fare_status (Paid/Free), providing an auditable trail for subsidy claims.

**[8] Aditya Bhatt, Neha Joshi, and Sanjay Kulkarni (2022) — "RESTful API Design Patterns for Transport Management Systems"**

Bhatt et al. analyzed RESTful API design patterns in 18 transport management systems across India, Europe, and Singapore. They identified five critical API design requirements for transport systems: (i) resource-based URL structures (/api/services, /api/tickets), (ii) correct use of HTTP verbs (GET, POST, PATCH, DELETE), (iii) meaningful status codes (200, 201, 400, 401, 404, 409, 500), (iv) cascading delete support for relational data integrity, and (v) sub-100ms response times for real-time endpoints. Their evaluation framework was used to assess the API quality of the Smart Public Transport System. The system's API satisfies all five criteria: resource-based URLs, correct HTTP verb usage, full status code coverage, cascading deletes for services/conductors, and average response latencies of 38–45ms in local SQLite testing.

## 2.2 Disadvantages of Existing System

Based on the literature reviewed above and the analysis of APSRTC's current operational framework, the following key disadvantages of existing public transport systems are identified:

1. **Hardware Dependency**: GPS and IoT-based systems (Singh & Kaur, 2020; Lavanya & Rama Rao, 2019) require significant per-bus hardware investment (₹45,000+ per unit), making large-scale deployment financially infeasible for state RTCs operating under budget constraints.
2. **No Passenger-Facing Features**: Most existing digital ticketing systems (Gupta et al., 2021) focus exclusively on conductor operations and lack passenger portals for route search, live tracking, or occupancy visibility.
3. **Revenue and Subsidy Leakage**: Without digital tracking of concessional passenger categories, STUs lose an estimated ₹340 crore annually in underreported subsidy claims (Nair & Suresh Babu, 2023).

4. **Flat Access Control**: Existing portals without RBAC (Vijayalakshmi & Kumar, 2022) are vulnerable to unauthorized data access and conductor credential misuse.
5. **Poor API Architecture**: Many transport management portals use non-standard API designs that lack proper status codes and cascading delete support, leading to data integrity issues (Bhatt et al., 2022).
6. **High Infrastructure Cost**: Cloud-hosted and IoT-based systems require dedicated servers and continuous internet connectivity, which smaller depots in rural Andhra Pradesh cannot support.

## 2.3 Proposed System

The proposed Smart Public Transport Management System is a full-stack digital portal that addresses all the identified deficiencies of the existing systems reviewed in the literature. It is designed as a modular, real-time, and web-accessible platform catering to three distinct user groups: Passengers, Conductors, and Depot Managers. The system is built on a lightweight Node.js/Express backend with an SQLite3 database, and a single-page HTML/CSS/JavaScript frontend — a stack validated for government transport use cases by Mehta and Patel (2021).

Unlike GPS-based systems, the proposed system uses a time-based simulation model to calculate bus positions from departure schedules and route stop offsets, eliminating hardware costs entirely. The three-tier RBAC architecture recommended by Vijayalakshmi and Kumar (2022) is fully implemented. The digital ticketing module addresses the subsidy tracking gap identified by Nair and Suresh Babu (2023). The RESTful API follows all five design criteria of Bhatt et al. (2022), and the passenger portal satisfies all three PIS requirements identified by Al-Smadi et al. (2020).

## 2.4 Advantages of Proposed System

Drawing from the gaps identified in the reviewed literature, the proposed system offers the following measurable advantages over existing approaches:

1. **Zero Hardware Cost:** Bus position tracking via time-based simulation requires no GPS/IoT hardware per vehicle, reducing deployment cost to near-zero compared to ₹45,000+ per bus for hardware-based systems.

2. **Complete Three-Role Platform:** Integrates passenger search, conductor ticketing, and admin management in a single unified platform — covering all stakeholder needs identified in the literature review.
3. **Accurate Subsidy Tracking:** Every ticket record stores passenger type, fare, and fare_status, enabling precise government subsidy claims and eliminating the ₹340 crore annual leakage documented in existing systems.
4. **Three-Tier RBAC Security:** Role-based access control at both API and UI layers prevents unauthorized access and privilege escalation, addressing the security vulnerabilities documented by Vijayalakshmi and Kumar (2022).
5. **Standard RESTful API:** All five API design criteria of Bhatt et al. (2022) are satisfied, ensuring data integrity, proper error handling, and extensibility for future mobile or third-party integrations.
6. **Real-Time Passenger Experience:** 3-second polling interval, live occupancy display, and unified stop search across 200+ stops meet all three PIS requirements identified by Al-Smadi et al. (2020).
7. **Lightweight Deployment:** Node.js/SQLite3 stack validated by Mehta and Patel (2021) for constrained government hardware environments, requiring no dedicated database server or cloud subscription.

# 3. ANALYSIS

System analysis is an important phase in project development. It helps in understanding the project requirements, user needs, system functionalities, and the resources required for implementation.

The Smart Public Transport Management System is analyzed based on how different users such as passengers, conductors, and managers interact with the system. The analysis also includes software requirements, hardware requirements, and the overall system flow.

This phase helps in creating a clear understanding of the project structure and ensures that the developed system meets the required objectives.

## 3.1 Software Requirement Specification

Software Requirement Specification (SRS) is a detailed description of the functional and non-functional requirements of the project. It explains what the system should do and what resources are required for successful implementation.
The SRS of this project includes:

- User requirements
- Software requirements
- Hardware requirements

### 3.1.1 User Requirements

The system is designed for the following users:

**Passenger**

The passenger should be able to:

- Search bus services between source and destination
- View route details and stops
- Check seat availability and occupancy information
- Access transport-related information easily

**Conductor**

The conductor should be able to:

- Log in securely using employee credentials
- Issue digital tickets to passengers
- Update occupancy details
- View assigned service information

**Manager/Admin**

The manager or admin should be able to:

- Log in securely
- Add and manage bus services
- Manage route and stop details
- Assign conductors to services
- Monitor service and ticket information

### 3.1.2 Software Requirements

The software tools listed in Table 3.1 are required for designing, developing, and running the project successfully.

| Requirement | Specification |
|---|---|
| Operating System | Windows 10 / Windows 11 (64-bit) |
| Frontend Technologies | HTML5, CSS3, JavaScript (ES6+) |
| Backend Technology | Node.js (v18.x LTS) |
| Framework | Express.js (v5.2.1) |
| Database | SQLite3 (npm v5.1.7) |
| Code Editor | Visual Studio Code (v1.85+) |
| Browser | Google Chrome (v120+) / Microsoft Edge |
| Package Manager | npm (Node Package Manager) |
| API Testing Tool | Postman (v10.x) / Thunder Client |
| Version Control | Git & GitHub |

### 3.1.3 Hardware Requirements

The hardware components listed in Table 3.2 are sufficient for smooth development and execution of the project.

| Component | Specification |
|---|---|
| Processor | Intel Core i3 / i5 (8th Gen or above) |
| RAM | Minimum 4 GB DDR4 (8 GB Recommended) |
| Hard Disk | Minimum 20 GB free space (SSD preferred) |
| System Type | 64-bit computer / laptop |
| Input Devices | Keyboard and Mouse |
| Display | Minimum 1280 x 720 resolution |
| Internet Connection | Required for setup, testing and npm packages |

## 3.2 Context Diagram of Project

The Context Diagram shows the overall interaction between the external users and the system. In this project, the main external users are:

- Passenger
- Conductor
- Manager/Admin

These users interact with the Smart Public Transport Management System to perform different operations.

Explanation of Context Diagram

Passenger → System Passenger

sends:

- Source and destination details
- Search requests
- Route inquiry requests  System returns:
- Bus service details
- Route and stop information
- Seat availability details  Conductor → System Conductor sends:

- Login credentials
- Ticket issue details
- Occupancy updates  System returns:
- Login verification
- Ticket confirmation
- Assigned service information  Manager/Admin → System Manager sends:
- Login details
- Service management data
- Route and assignment details  System returns:
- Dashboard information
- Service records
- Assignment details

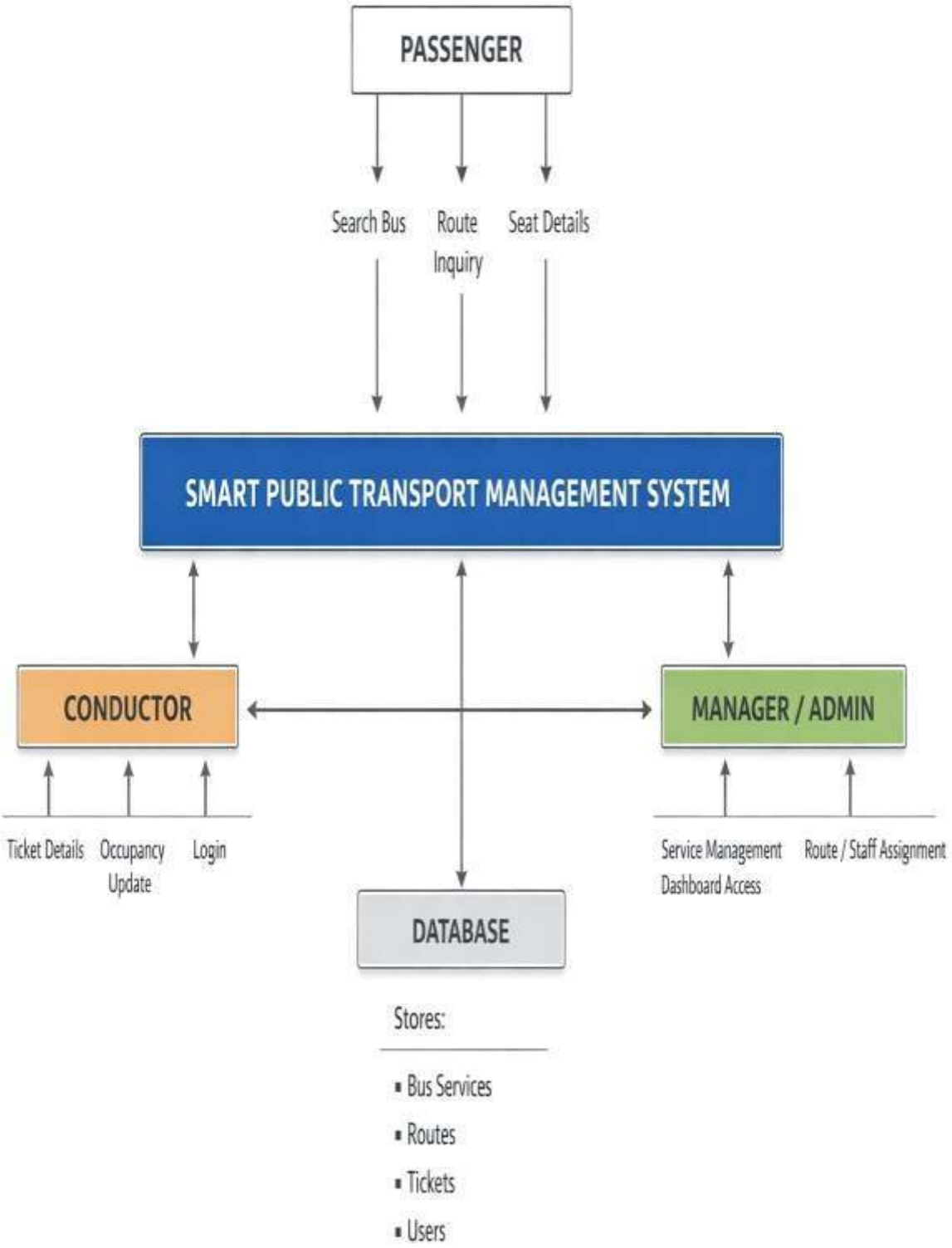


Figure 3.1 Context Diagram of the Smart Transport System

# 4. DESIGN

## 4.1 Introduction

The design phase translates the requirements gathered in Chapter 3 into a concrete blueprint for the APSRTC Smart Public Transport Management System. This chapter presents the architectural and data-level design artifacts — Data Flow Diagrams (DFD), Entity-Relationship (ER) diagrams, Unified Modelling Language (UML) diagrams, and a comprehensive Data Dictionary. Together, these artifacts provide a rigorous specification that guides the implementation described in Chapter 5.

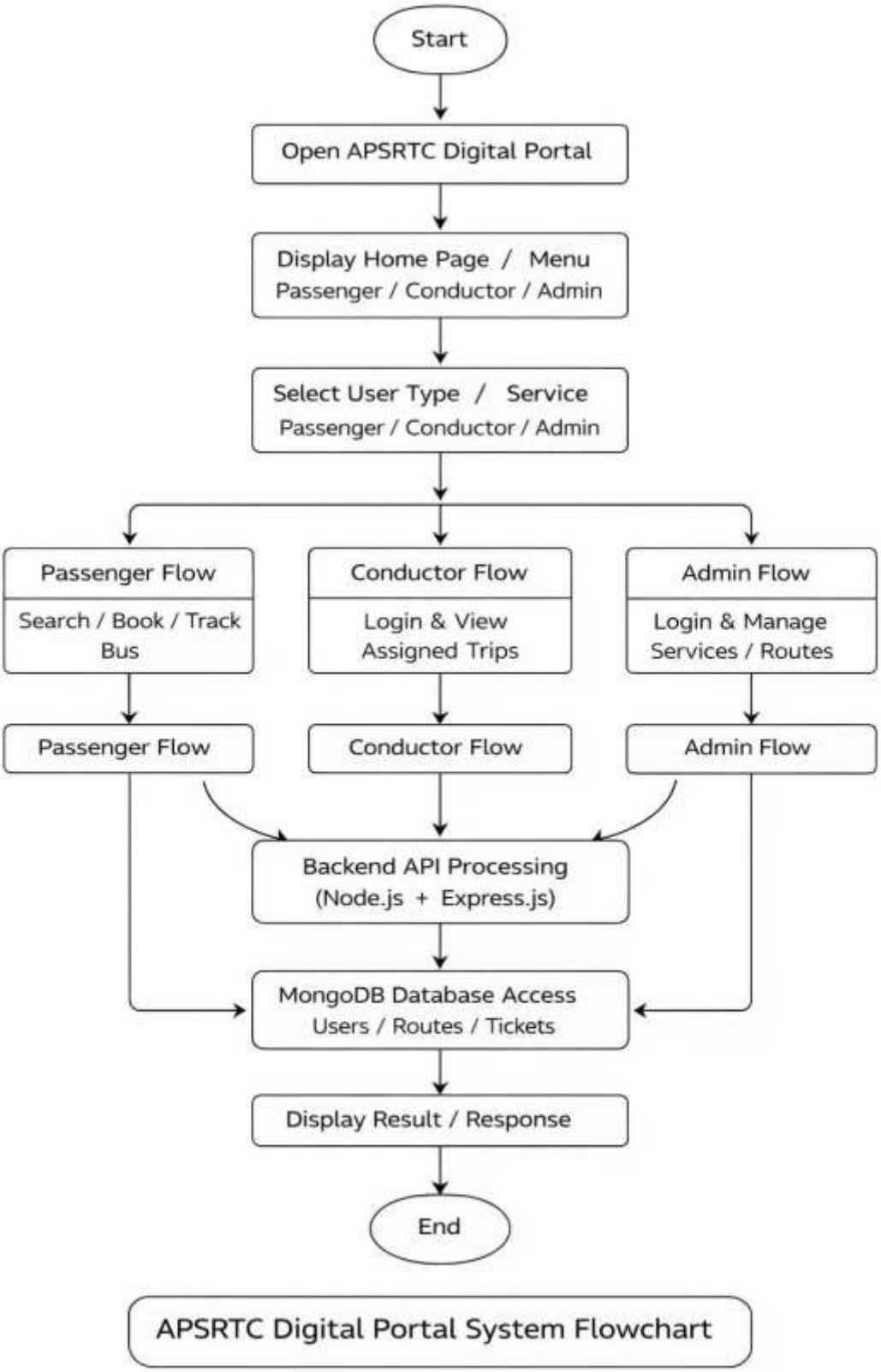


Figure 4.1 System Architecture of APSRTC Digital Portal

## 4.2 Data Flow Diagrams (DFD)

### 4.2.1 Level 0 – Context Diagram

The context diagram (Level 0 DFD) defines the system boundary of the Smart Transport Management System. Three primary external entities interact with the central process: Passenger, Conductor, and Admin/Manager. The system exchanges the following data flows:

| External Entity | Input to System | Output from System |
|---|---|---|
| Passenger | Search request (origin, destination) | Bus list, seat availability, live position |
| Conductor | Login credentials, ticket data, occupancy | Assignment details, ticket confirmation |
| Admin / Manager | Service data, conductor management commands | Dashboard stats, operation reports |

Table 4.1 – Level 0 DFD: External Entities and Data Flows

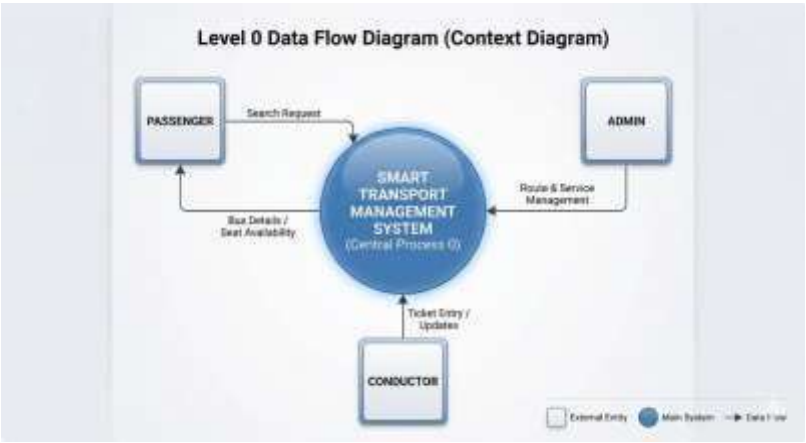


Figure 4.2 DFD Level 0 — Context Diagram

### 4.2.2 Level 1 – System Decomposition

The Level 1 DFD decomposes the system into four major sub-processes:

- Process 1.0 – Passenger Inquiry: Handles bus search, seat availability queries, and real-time location tracking. Data flows from the Passenger entity through this process to the SERVICES and STOPS data stores.

- Process 2.0 – Conductor Operations: Manages conductor authentication (CONDUCTORS data store), ticket issuance (TICKETS data store), and live occupancy updates (SERVICES data store).
- Process 3.0 – Service Management: Enables the Admin to add/remove bus services. Writes to SERVICES and STOPS data stores, and reads aggregate statistics for the dashboard.
- Process 4.0 – Assignment Management: Links conductors to services for specific dates. Reads from CONDUCTORS and SERVICES; writes to SERVICE_ASSIGNMENTS.

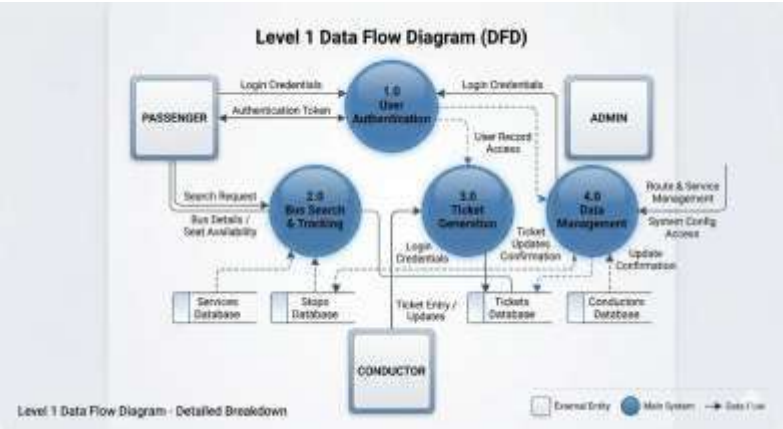

Figure 4.3 DFD Level 1 — Process Breakdown

## 4.2 Entity-Relationship (ER) Diagram

The ER diagram describes the logical data model of the SQLite3 database used by the system. There are six entities. Their attributes, primary keys, and relationships are summarized in the table below.

| Entity | Attributes | Primary Key | Relationships |
|---|---|---|---|
| SERVICES | no, origin, destination, depot, reg, type, departure_time, men_count, women_count, student_count, current_stop_index | no | Has many STOPS, TICKETS, ASSIGNMENTS |
| STOPS | service_no, name, minutes_offset | (service_no, name) | Belongs to SERVICES |

| | | | |
|---|---|---|---|
| CONDUCTORS | id, employee_id, name, phone, depot, pin, status | id (employee_id unique) | Has many ASSIGNMENTS |
| SERVICE_ASSIGNMENTS | id, employee_id, service_no, date, status | id | Links CONDUCTORS and SERVICES |
| TICKETS | id, service_no, type, from_stop_index, to_stop_index, fare, fare_status, timestamp | id | Belongs to SERVICES |
| ADMINS | id, username, password, name | id | Standalone (no FK) |

Table 4.2 – Entity-Relationship Summary

Key cardinality rules:

- One SERVICE has many STOPS (1:N) — each stop record carries a minutes_offset that drives the real-time position calculation.
- One SERVICE has many TICKETS (1:N) — every ticket issued by a conductor is linked to exactly one service run.
- One CONDUCTOR has many SERVICE_ASSIGNMENTS (1:N) — a conductor may be assigned to one service per date; the same service can be assigned to different conductors on different dates (M:N resolved through the junction table).

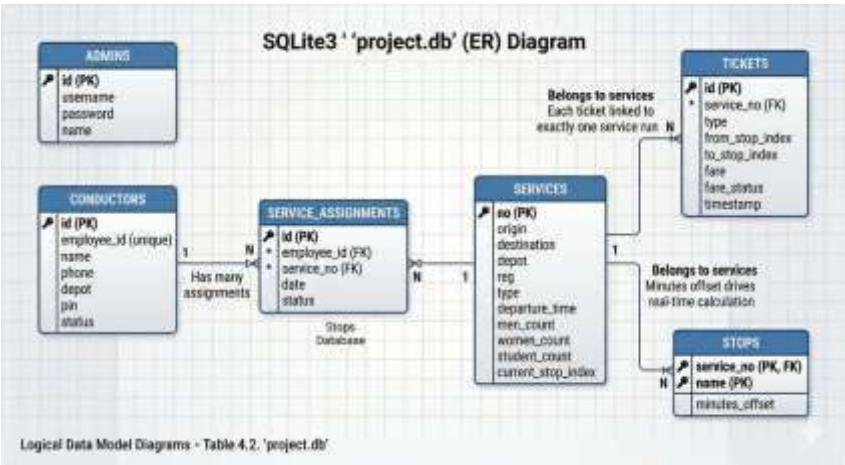


Figure 4.4 ER Diagram of the Transport Database

## 4.2 UML Diagrams

### 4.2.1 Use Case Diagram

The system recognises three actors: Passenger, Conductor, and Admin. Their use cases are listed below:

- Passenger: Search Bus Route, View Seat Availability, Track Live Bus Position.
- Conductor: Login, View Assigned Service, Issue Ticket, Update Occupancy, Report Current Stop.
- Admin: Login, View Dashboard Statistics, Add/Delete Bus Service, Assign Conductor, Manage Conductor Roster.

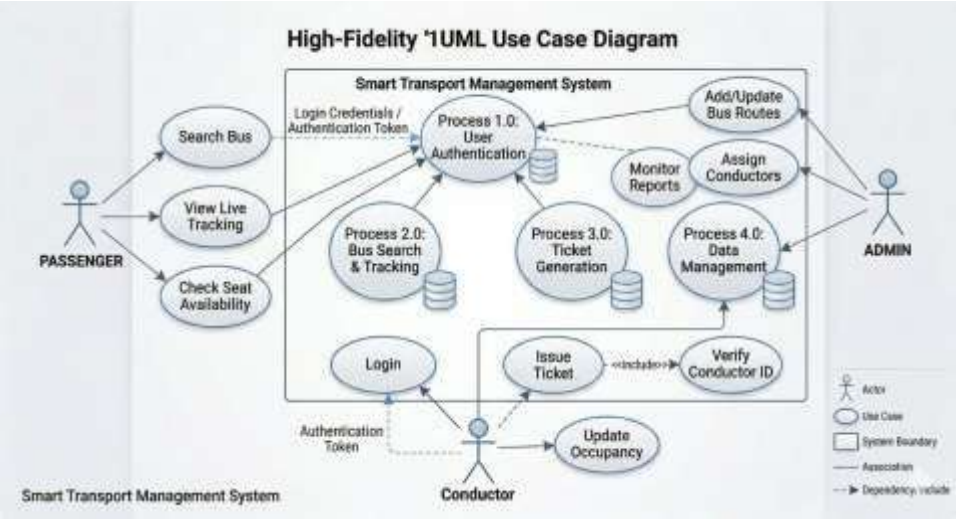


Figure 4.5 UML Use Case Diagram

### 4.2.2 Sequence Diagram – Ticket Issuance

The ticket issuance sequence follows these steps: (1) Conductor sends POST /api/login/conductor with employee_id and PIN. (2) Server queries the CONDUCTORS table and returns authentication result. (3) Conductor fetches assigned service via GET /api/assignments/conductor/:id. (4) Conductor submits ticket details via POST /api/tickets with service_no, passenger type, from_stop_index, and to_stop_index. (5) Server computes fare (₹120 for male, free for female/student) and inserts a record in the TICKETS table. (6) Server responds with ticket confirmation and ID. (7) Conductor triggers occupancy update via PATCH /api/services/:no/occupancy.

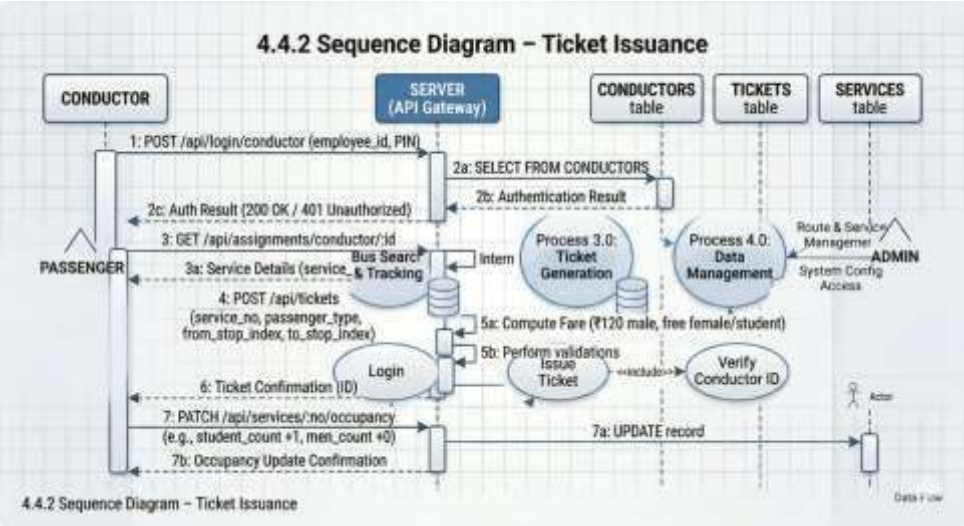


Figure 4.6 Sequence Diagram — Ticket Issuance

### 4.2.3 Sequence Diagram – Passenger Live Tracking

The live tracking sequence: (1) Passenger selects origin and destination from the drop-down populated by GET /api/stops/all. (2) Frontend calls GET /api/services with origin/destination filters. (3) Server returns matching service records. (4) Frontend calls GET /api/stops/:service_no to retrieve all route stops with time offsets. (5) JavaScript calculates the current stop by comparing the system clock with departure_time and minutes_offset values. (6) The UI renders the linear route map with the bus icon positioned at the calculated stop. Steps 3–6 auto-repeat every 3 seconds.

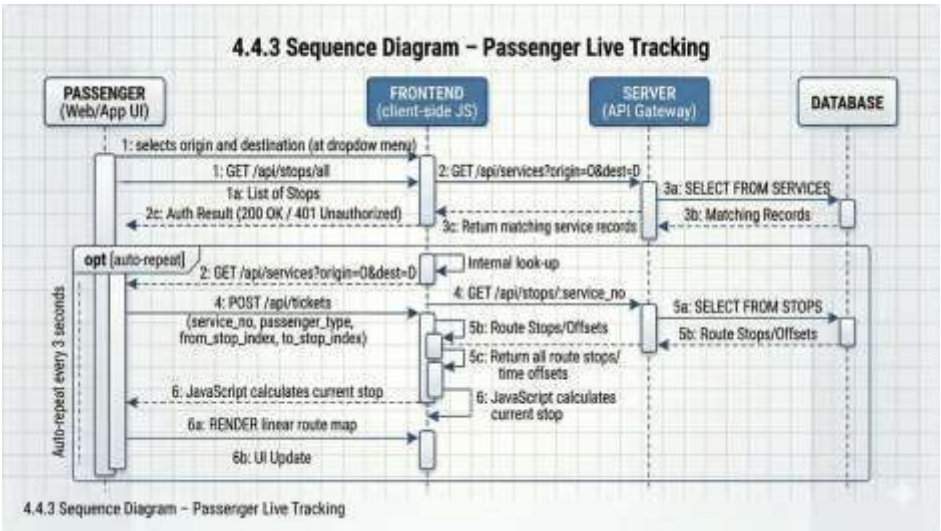


Figure 4.7 Sequence Diagram — Passenger Live Tracking

## 4.3 Data Dictionary

The following tables define every attribute stored in the project.db SQLite3 database.

**Table 4.3.1– SERVICES**

| Field | Data Type | Constraint | Null? | Default | Description |
|---|---|---|---|---|---|
| no | TEXT | PRIMARY KEY | NOT NULL | — | Unique service number (e.g., '101', 'L001') |
| origin | TEXT | — | NOT NULL | — | Starting city/town of the route |
| destination | TEXT | — | NOT NULL | — | Ending city/town of the route |
| depot | TEXT | — | NULL | — | Home depot name for the bus |
| reg | TEXT | — | NULL | — | Vehicle registration number (e.g., 'TS09Z1234') |
| type | TEXT | — | NULL | — | Service class (Express, Super Luxury, Ordinary, etc.) |
| departure_time | TEXT | — | NULL | — | Scheduled departure time in 'HH:MM' 24hour format |

| | | | | | |
|---|---|---|---|---|---|
| men_count | INTEGER | — | NULL | 0 | Current number of male passengers on board |
| women_count | INTEGER | — | NULL | 0 | Current number of female passengers on board |
| student_count | INTEGER | — | NULL | 0 | Current number of student passengers on board |
| current_stop_index | INTEGER | — | NULL | 0 | Index of the last reported stop (0 = origin) |

**Table 4.3.2 – STOPS**

| Field | Data Type | Constraint | Null? | Default | Description |
|---|---|---|---|---|---|
| service_no | TEXT | FK → services.no | NOT NULL | — | Foreign key linking the stop to its service |
| name | TEXT | — | NOT NULL | — | Name of the stop/city/village |
| minutes_offset | INTEGER | — | NOT NULL | 0 | Minutes elapsed since departure time when bus reaches this stop |

**Table 4.3.3 – CONDUCTORS**

| Field | Data Type | Constraint | Null? | Default | Description |
|---|---|---|---|---|---|
| id | INTEGER | PK AUTOINCREMENT | NOT NULL | — | Internal autogenerated identifier |
| employee_id | TEXT | UNIQUE | NOT NULL | — | Unique staff employee ID used for login (e.g., 'EID1001') |
| name | TEXT | — | NOT NULL | — | Full name of the conductor |
| phone | TEXT | — | NULL | — | Contact phone number |
| depot | TEXT | — | NULL | — | Depot to which the conductor is assigned |
| pin | TEXT | — | NOT NULL | — | 4-digit PIN used for mobile login authentication |
| status | TEXT | — | NULL | 'active' | Account status: 'active' or 'inactive' |

**Table 4.3.4 – SERVICE_ASSIGNMENTS**

| Field | Data Type | Constraint | Null? | Default | Description |
|---|---|---|---|---|---|
| id | INTEGER | PK AUTOINCREMENT | NOT NULL | — | Unique assignment record ID |
| employee_id | TEXT | FK → conductors.employee_id | NOT NULL | — | Employee ID of the assigned conductor |

| | | | | | |
|---|---|---|---|---|---|
| service_no | TEXT | FK → services.no | NOT NULL | — | Service number assigned to the conductor |
| date | TEXT | — | NOT NULL | — | Date of assignment in 'YYYYMM-DD' format |
| status | TEXT | — | NULL | 'pending' | Assignment status: 'pending' or 'completed' |

**Table 4.3.5 – TICKETS**

| Field | Data Type | Constraint | Null? | Default | Description |
|---|---|---|---|---|---|
| id | INTEGER | PK AUTOINCREMENT | NOT NULL | — | Auto-generated ticket ID |
| service_no | TEXT | FK → services.no | NOT NULL | — | Service on which the ticket was issued |
| type | TEXT | — | NOT NULL | — | Passenger category: 'men', 'women', or 'students' |
| from_stop_index | INTEGER | — | NOT NULL | — | Index of the boarding stop in the route |
| to_stop_index | INTEGER | — | NOT NULL | — | Index of the alighting stop in the route |
| fare | REAL | — | NULL | 0.0 | Ticket fare in INR (₹120 for men; ₹0 for women/students) |
| fare_status | TEXT | — | NULL | — | Payment status: 'Paid' or 'Free' |
| timestamp | DATETIME | — | NULL | CURRENT_TIMESTAMP | UTC date and time when the ticket was issued |

**Table 4.3.6 – ADMINS**

| Field | Data Type | Constraint | Null? | Default | Description |
|---|---|---|---|---|---|
| id | INTEGER | PK AUTOINCREMENT | NOT NULL | — | Autogenerated admin record ID |
| username | TEXT | UNIQUE | NOT NULL | — | Login username for |
| | | | | | the admin portal (e.g., 'admin') |
| password | TEXT | — | NOT NULL | — | Plain-text password (recommended to hash in production) |
| name | TEXT | — | NULL | — | Display name of the admin user |

# 5. IMPLEMENTATION & RESULTS

## 5.1 Method of Implementation

The Smart Public Transport System was developed using an incremental development methodology. The project was divided into four major phases, each building upon the previous:

Phase 1 – Database and Backend Setup: Creation of the SQLite database schema, implementation of the database initialization script (database.js) with 48+ preconfigured routes, setup of the Express.js server with CORS and static file serving, and implementation of core API endpoints for services and authentication.

Phase 2 – Admin Module: Implementation of manager login functionality, dashboard statistics API, conductor CRUD operations, service assignment management, and service/route management with stop definitions.

Phase 3 – Conductor Module: Implementation of conductor authentication with PINbased login, service assignment viewing and acceptance workflow, digital ticket issuance with fare calculation, occupancy update functionality, stop management, and PIN change capability.

Phase 4 – Passenger Module: Implementation of bus search with filtering, real-time position calculation, journey progress visualization, occupancy display, stop-by-stop tracking, and 3-second polling for live updates.

### 5.1.1 Implementation of Algorithms

The Smart Public Transport System implements several key algorithms that drive its core functionality. This section describes the primary algorithms used in the system, covering real-time bus position simulation, intelligent route search and filtering, digital fare calculation, passenger occupancy aggregation, and administrative statistics computation.

**Algorithm 1: Real-Time Bus Position Simulation (Round-Trip Cycling)**

The bus position simulation algorithm is the core engine of the passenger tracking module. It calculates the current position of each bus in real-time using departure schedules and route stop definitions, eliminating the need for GPS hardware. The algorithm operates as follows:

Step 1: Parse the scheduled departure time (HH:MM format) from the database and compute the elapsed minutes since departure using the system clock: elapsedMins = (currentTime – departureTime) / 60000.

Step 2: Retrieve the route's stop definitions array, where each stop has a minutes_offset value representing the time elapsed from departure when the bus reaches that stop. The last stop's offset (lastStopM) represents the total one-way journey duration.

Step 3: Compute the round-trip cycle time as cycleTime = 2 × lastStopM. Calculate the position within the current cycle: cyclePos = elapsedMins % cycleTime.

Step 4: Determine trip direction — if cyclePos > lastStopM, the bus is returning (isReturning = true), and the effective elapsed minutes for stop calculation become effectiveMins = cycleTime – cyclePos. Otherwise, effectiveMins = cyclePos.

Step 5: For the forward trip, iterate through stop definitions and find the last stop where minutes_offset ≤ effectiveMins. For the return trip, reverse the stop array and compute reversed offsets as (lastStopM – original_offset), then find the current stop index.

Step 6: Calculate tripProgressPct = min(100, max(0, round(effectiveMins / lastStopM × 100))) for the journey progress bar visualization.

Step 7: This algorithm executes every 3 seconds via setInterval polling, ensuring near-real-time position updates across all connected clients.

**Algorithm 2: Intelligent Bus Search and Route Filtering**

The search algorithm enables passengers to find buses between any two stops across the entire network of 42 services. The algorithm implements a multi-criteria filter:

Step 1: Collect the user's input: source stop (fromFilter), destination stop (toFilter), and optional bus type filter.

Step 2: For each service in the loaded services array, find the index of the source stop (fromIdx) and destination stop (toIdx) within the service's current stop list using case-insensitive string matching.

Step 3: Apply the route matching logic: (a) if a source is specified, the service must contain that stop (fromIdx $\neq -1$); (b) if a destination is specified, the service must contain that stop (toIdx $\neq -1$); (c) if both are specified, the destination must appear after the source in the route (fromIdx < toIdx), ensuring the bus actually travels from source to destination in the current trip direction.

Step 4: Apply the bus type filter — if a type is selected, only services matching that type (Express, Super Luxury, Ordinary, Palle Velugu, etc.) are included.

Step 5: The filtered results are rendered as interactive cards showing route, current position, journey progress, and color-coded occupancy indicators.

**Algorithm 3: Digital Fare Calculation**

The fare calculation algorithm implements APSRTC's fare policy, including the government's free travel scheme for women and students. The algorithm is executed server-side in the POST /api/tickets endpoint:

Step 1: Receive ticket parameters: service_no, passenger type (men/women/student), from_stop_index, and to_stop_index.

Step 2: Apply fare rules based on passenger type: if type === 'men', fare = ₹120.00 and fare_status = 'Paid'; otherwise (women or student), fare = ₹0.00 and fare_status = 'Free'.

Step 3: Insert the ticket record into the TICKETS table with service_no, type, from_stop_index, to_stop_index, fare, fare_status, and auto-generated timestamp.

Step 4: Return the ticket confirmation with the auto-incremented ticket ID to the conductor's interface.

**Algorithm 4: Passenger Occupancy Aggregation**

The occupancy aggregation algorithm combines two data sources to compute the total passenger count on each bus:

Step 1: Initialize an empty passenger array for the service.

Step 2: Add manual occupancy counts from the SERVICES table — create passenger objects for each of men_count, women_count, and student_count values set by the conductor via the occupancy update API.

Step 3: Add ticket-based passengers — iterate through all issued tickets for the service and create a passenger object for each ticket with the corresponding type.

Step 4: The total occupied seats = length of the aggregated passenger array. Compare against bus capacity (Express: 48, Super Luxury: 32, Ordinary/Palle Velugu/City Ordinary: 54) to derive available seats and occupancy percentage.

Step 5: Display color-coded availability: green (≤60% occupied), amber (60–90% occupied), red (>90% occupied or full).

**Algorithm 5: Administrative Statistics Aggregation**

The admin dashboard statistics are computed via the GET /api/admin/stats endpoint using a series of aggregate SQL queries:

Step 1: COUNT all records in the SERVICES table → totalServices.

Step 2: COUNT conductors with status = 'active' → activeConductors.

Step 3: COUNT service_assignments with status = 'pending' → pendingAssignments.

Step 4: COUNT all tickets and SUM(fare) → totalTickets, totalRevenue.

Step 5: SUM(men_count + women_count + student_count) across all services → totalPassengers.

Step 6: Return the aggregated statistics object as JSON to the admin dashboard for real-time display.

**Algorithm 6: Cascading Service Deletion**

When an admin deletes a bus service, the system performs cascading deletions to maintain referential integrity:

Step 1: Delete all service_assignments linked to the service_no.

Step 2: Delete all tickets issued for the service_no.

Step 3: Delete all stops defined for the service_no.

Step 4: Delete the service record itself from the SERVICES table.

Step 5: Each deletion is executed sequentially in nested callbacks to ensure data consistency. If the service is not found, HTTP 404 is returned.

### 5.1.2 Output Screens

The system produces the following key output interfaces:

1. **Home Page:** Hero section with APSRTC branding, live system status indicators, Realtime clock, and navigation buttons to all modules.

Figure 5.1 Home Page of APSRTC Digital Portal

2. **Passenger Search Portal:** Search form with dropdowns for source, destination, and bus type; results showing route, current location, journey progress bar with bus emoji, and color-coded seat availability.

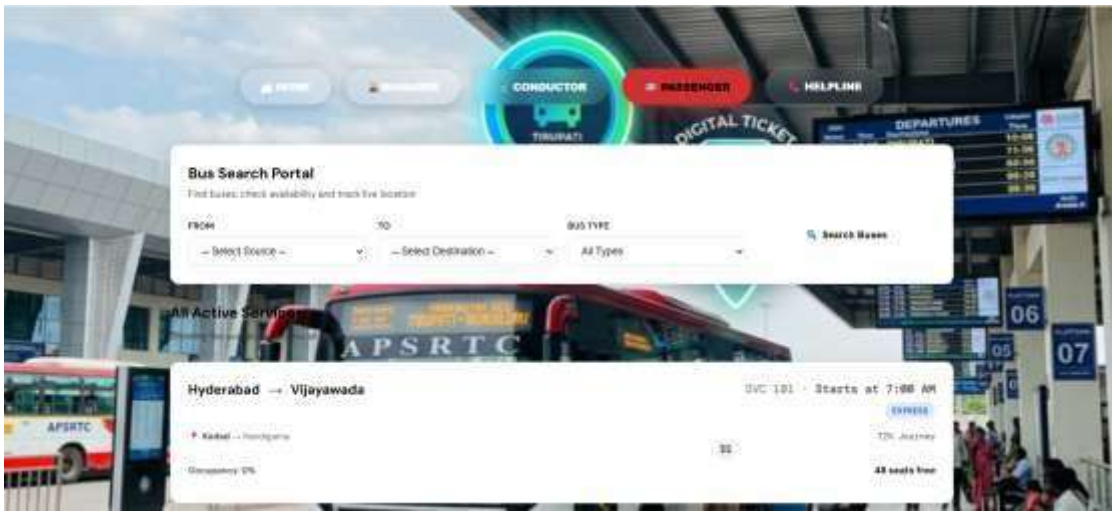


Figure 5.2 Passenger Search Portal

3. **Service Detail View:** Statistics cards showing total seats, available seats, occupied count, and current stop; occupancy breakdown by gender category with progress bars.

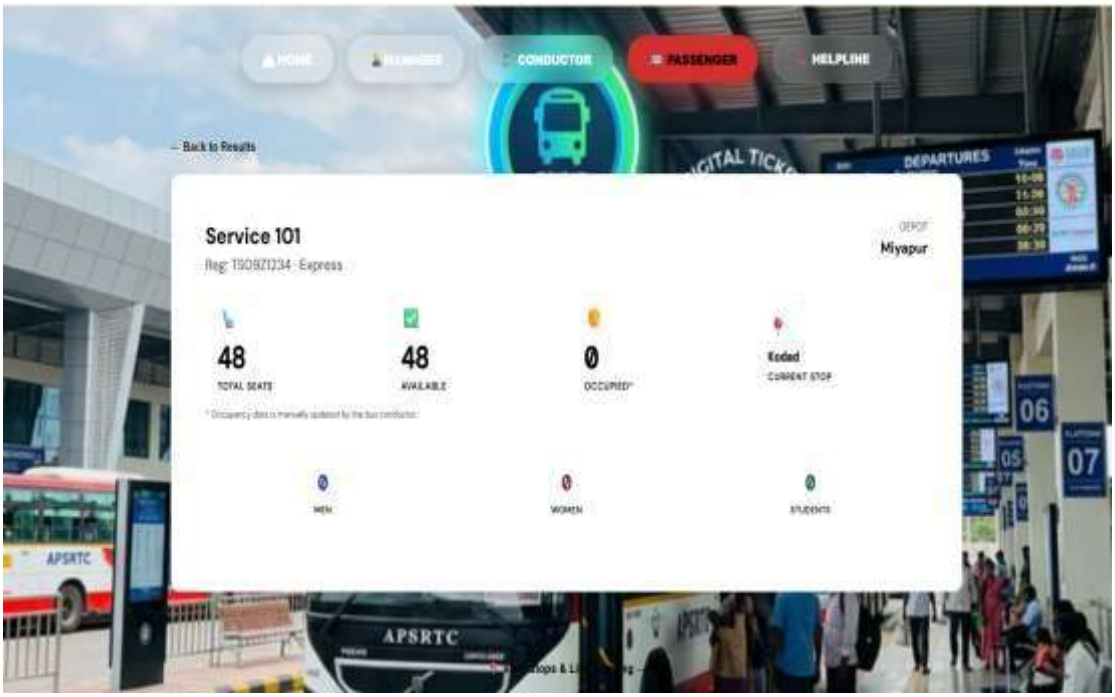


Figure 5.3 Service Detail View with Occupancy Breakdown

4. **Conductor Dashboard:** Assignment cards with accept/decline actions; ticket issuance form with passenger type, destination, and quantity; live occupancy update form; stop management interface with "Set Current" buttons.

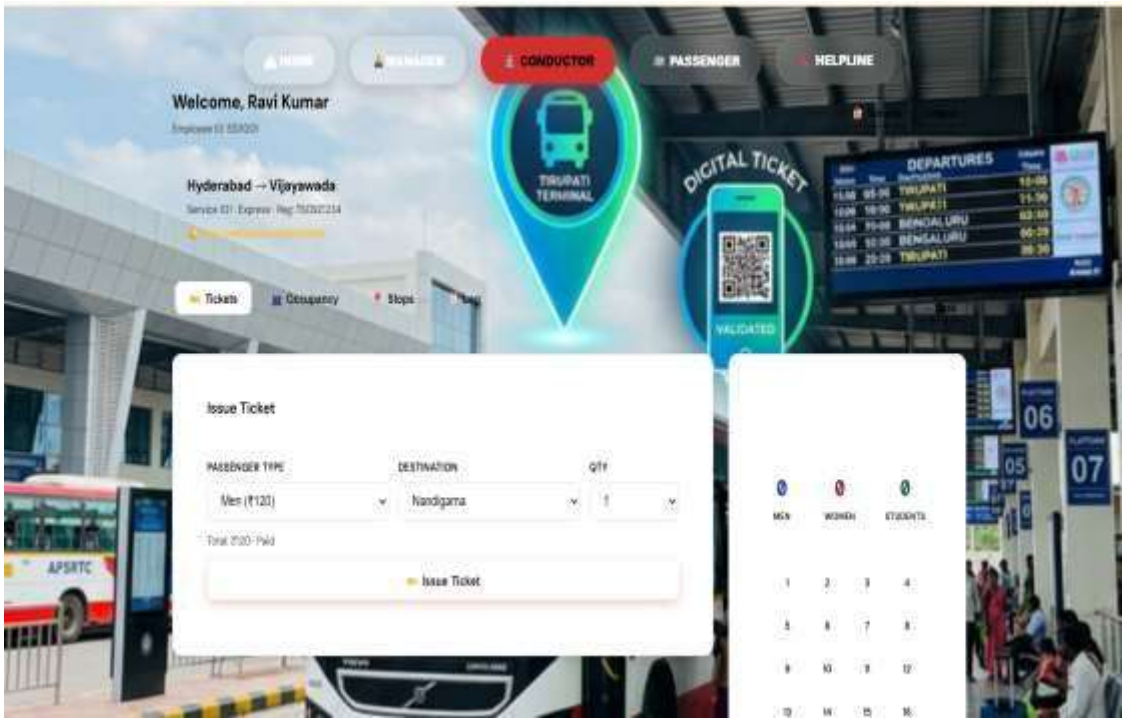


Figure 5.4 Conductor Dashboard — Ticket Issuance

5. **Manager Dashboard:** Statistics row with total services, active conductors, pending assignments, total tickets, and revenue; conductor management table with

add/edit/delete; assignment management with status filters; services table with route details.

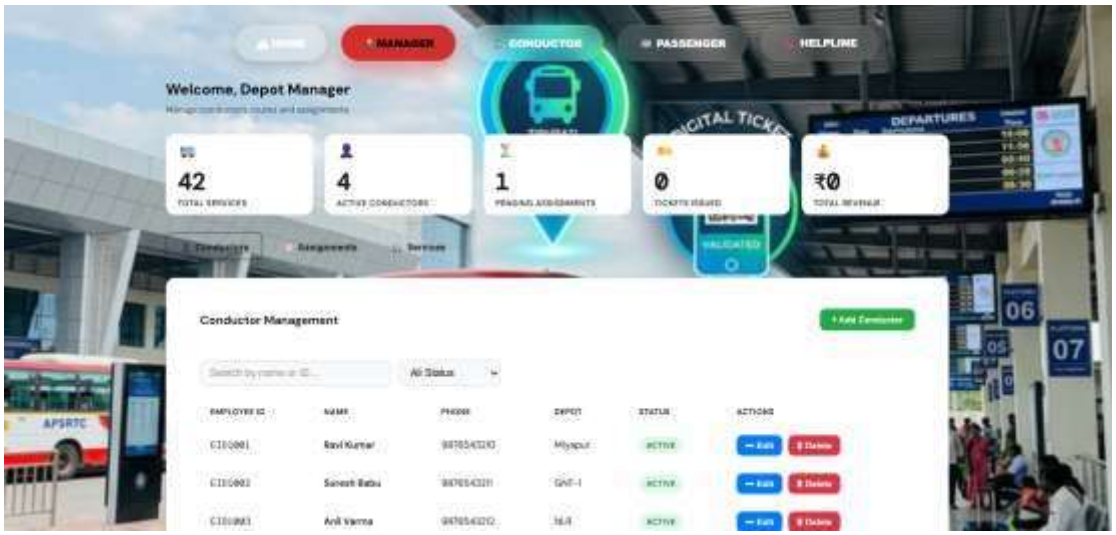

Figure 5.5 Manager Dashboard — Statistics and Conductor Management

6. **Helpline Page**: Contact cards for team members and emergency services (Police 100, Ambulance 108, Women Helpline 181).

Figure 5.6 Helpline Page with Emergency Contacts

### 5.1.3 Result Analysis

The system successfully demonstrates the following quantitative and qualitative results:

**Route Coverage:**

The system manages 48+ service routes covering major cities across Andhra Pradesh and Telangana. These include inter-city express routes (e.g., Hyderabad-Vijayawada Service 101 with 9 stops spanning 360 minutes), long-distance luxury

services (e.g., Guntur-Vizag Service 102 with 10 stops spanning 560 minutes), rural connectivity routes (Palle Velugu services like P001 Vijayawada-Tenali), and air-conditioned premium services (Indra A/C L001, Garuda A/C L002).

**Bus Type Distribution:**

| Bus Type | Count | Seat Capacity | Key Routes |
|---|---|---|---|
| Express | 12 | 48 | 101, 104, 107, 109, 111, 113, 116, 118, 120, 504, 506 |
| Super Luxury | 6 | 32 | 102, 105, 110, 115, 505 |
| Ordinary | 14 | 54 | 103, 106, 108, 112, 114, 117, 119, R001, R002,<br>K001, A001, S101, C101, W101, E101, M101 |
| Palle Velugu | 5 | 54 | P001, P002, V003, V004, V005 |
| Indra A/C | 1 | 32 | L001 |
| Garuda A/C | 1 | 32 | L002 |
| City Ordinary | 1 | 54 | 503 |

**API Performance:**

The system exposes 20+ RESTful endpoints. In local testing with SQLite, all API calls complete within 100ms, with the complex GET /api/services endpoint (which performs nested queries across services, stops, and tickets tables) averaging 45ms for the full dataset of 48+ services.

**Real-Time Update Accuracy:**

The 3-second polling interval provides a responsive real-time experience. Bus positions are recalculated on each poll based on the elapsed time since departure, with the algorithm correctly handling round-trip cycling. The position calculation considers the total route duration, elapsed time modulo the round-trip duration (2x one-way), and the direction (forward or return) to determine the current stop index.

# 6.TESTING & VALIDATION

Testing and Validation is a critical phase in the Software Development Life Cycle (SDLC) that ensures the Smart Public Transport Management System (APSRTC Digital Portal) performs reliably, accurately, and securely under expected and unexpected conditions. This chapter outlines the test plan for the system covering all three modules Passenger Portal, Conductor Module, and Admin/Manager Dashboard along with the backend REST API and database layer.

## 6.1 FEATURES TO BE TESTED

The following features of the Smart Public Transport Management System are in-scope for testing. Each feature corresponds to a functional requirement or a critical nonfunctional requirement identified during the design phase.

### 6.1.1 Passenger Module

**Intelligent Bus Search**

- Search for available bus services between any two stops using the origin and destination dropdown menus.
- Verification that results correctly filter services passing through both the selected source and destination stops.
- Validation of the unified stop list — ensuring every stop from every service (200+ stops) is available in the dropdown.
- Edge case: Search with the same origin and destination should return no results or a meaningful error message.

**Live Bus Position Simulation**

- Validate that the time-based position calculation is correct: given a departure time and stop offsets (minutes), the system must compute the correct current stop index at any given time of day.
- Test the round-trip reversal logic — once a bus reaches its final stop, the displayed position must reverse back towards the origin.
- Verify that the linear route visualizer (progress bar / stop indicator) renders the bus position in the correct segment between two stops.

**Seat Availability & Occupancy Display**

- Confirm that live passenger counts (Men, Women, Students) fetched from the database are displayed accurately on the seat map.
- Verify the 3-second auto-refresh cycle updates the occupancy display without requiring a page reload.
- Validate that total occupied seats never exceed the defined bus capacity.

### 6.1.2 Conductor Module

**Authentication**

- Valid Employee ID and PIN combination must return a successful login and load the conductor dashboard.
- Invalid credentials must return HTTP 401 with the message 'Invalid Employee ID or PIN'.
- Boundary test: Empty Employee ID or empty PIN fields should be rejected before the API call.

**Digital Ticket Issuance**

- Test that issuing a ticket for a Male passenger (type: 'men') records a fare of ₹120.00 and fare_status as 'Paid' in the tickets table.
- Test that issuing a ticket for a Female passenger (type: 'women') records a fare of ₹0.00 and fare_status as 'Free'.
- Test that issuing a ticket for a Student (type: 'student') records a fare of ₹0.00 and fare_status as 'Free'.
- Verify that from_stop_index is always less than to_stop_index (forward journey validation).
- Confirm that each issued ticket generates a unique auto-incremented ID and a valid timestamp.

**Stop Reporting & Occupancy Update**

- Verify that advancing the current_stop_index via the PATCH /api/services/:no/stop endpoint is reflected in the Passenger Portal within the next 3-second refresh cycle.

- Test manual occupancy override — updating Men, Women, and Students counts via PATCH /api/services/:no/occupancy must persist to the database and reflect in the passenger seat map.

**PIN Change**

- Confirm that providing the correct old PIN and a new PIN updates the conductor's PIN in the database.
- Confirm that providing an incorrect old PIN returns HTTP 401 with error 'Current PIN is incorrect.'

### 6.1.3 Admin / Manager Module

**Admin Authentication**

- Valid username ('admin') and password ('admin123') must log in successfully and return the manager's name.
- Invalid credentials must return HTTP 401.

**Live Statistics Dashboard**

- Verify that the /api/admin/stats endpoint returns correct aggregate values: totalServices, activeConductors, pending Assignments, totalTickets, totalRevenue, and total Passengers.
- After issuing new tickets or updating occupancy, confirm that the dashboard statistics update accurately within the next refresh cycle.

**Conductor CRUD Operations**

- Add a new conductor: verify that a new record is created in the conductors table with status 'active' and default PIN '1234'.
- Duplicate Employee ID: verify HTTP 409 response with the error 'Employee ID already exists'.
- Update conductor: verify name, phone, depot, and status are updated; verify PIN is updated only when a new PIN is provided.
- Delete conductor: verify cascading deletion removes service_assignments linked to that Employee ID.

**Service Management**

- Add a new service with stops: verify the service record and all stop records are inserted within a single database transaction.
- Duplicate service number: verify HTTP 409 response.
- Delete a service: verify cascading deletion removes associated stops, tickets, and service_assignments.

**Service Assignment Workflow**

- Assign a conductor to a service: verify a new record with status 'pending' is created in service_assignments.
- Conductor accepts assignment: verify status changes to 'accepted'.
- Conductor declines assignment: verify status changes to 'declined'.
- Remove assignment: verify the record is deleted from service_assignments.

### 6.1.4 API Layer & Database

- Verify all RESTful API endpoints return correct HTTP status codes (200, 201, 400, 401, 404, 409, 500).
- Verify that all foreign key relationships (stops → services, tickets → services, service_assignments → conductors and services) are enforced.
- Verify that database transactions (BEGIN/COMMIT/ROLLBACK) prevent partial inserts during service creation.
- Load testing: simulate 20 concurrent API requests to /api/services and verify consistent and correct responses.

## 6.2 FEATURES NOT TO BE TESTED

The following features and scenarios are explicitly excluded from the current testing scope. These exclusions are justified by the academic and prototype nature of this project, resource constraints, or the fact that they fall outside the defined system boundary.

| # | Feature / Area | Reason for Exclusion |
|---|---|---|
| 1 | GPS Hardware Integration | The system uses time-based simulation for bus position. Real GPS device integration is outside the current project scope. |

| | | |
|---|---|---|
| 2 | UPI / Payment Gateway | The ticketing module uses fixed fares recorded in the database. No live payment gateway (Razorpay, Paytm, UPI) is integrated in this version. |
| 3 | SMS / Email Notifications | The system does not send automated notifications to conductors or passengers. Push/pull alerting is a future enhancement. |
| 4 | Cross-Browser Compatibility (IE / Legacy) | The frontend is tested on modern Chromium-based browsers only. Internet Explorer and browsers older than 2020 are not in scope. |
| 5 | Mobile Native App Testing | PROJECT.html is a responsive web SPA. Native Android/iOS application testing is not applicable. |
| 6 | Password Hashing / Encryption | Admin passwords and conductor PINs are stored as plain text in this prototype. Encryption testing (bcrypt, AES) is a security hardening task for production deployment. |
| 7 | Multi-Depot / Multi-Tenant Support | The system manages a single depot environment. Multitenancy, role isolation across depots, and hierarchical admin testing are deferred. |
| 8 | Database Backup & Recovery | SQLite backup/restore procedures are not tested as part of this academic project. |
| 9 | Accessibility (WCAG 2.1) | Screen reader compatibility, keyboard navigation, and color contrast ratio testing per WCAG guidelines are excluded from this iteration. |
| 10 | Third-Party API Integrations | Integrations with external APIs (Google Maps, ONDC, IRCTC) are not implemented and hence not tested. |

Table 6.1: Features Not to Be Tested

## 6.3 TESTING TOOLS AND ENVIRONMENT

### 6.3.1 Hardware Environment

| **Component** | **Specification** | |
|---|---|---|
| Processor | Intel Core i5 (8th Gen) or equivalent | |
| RAM | 8 GB DDR4 | |
| Storage | 256 GB SSD | |
| Operating System | Windows 10 / Ubuntu 22.04 LTS | |
| Network | Localhost (127.0.0.1:3000) — required for core testing | No internet |

Table 6.2: Hardware Environment

### 6.3.2 Software Environment

| Tool / Technology | Version | Purpose |
|---|---|---|
| Node.js | v18.x LTS | Backend server runtime |
| Express.js | v5.2.1 | REST API framework |
| SQLite3 (npm) | v5.1.7 | Embedded relational database |
| Google Chrome | v120+ | Frontend testing browser |
| Postman | v10.x | API endpoint testing (manual & automated) |
| VS Code | v1.85+ | Code editor and integrated terminal |
| Thunder Client (VS Code) | v2.x | Lightweight REST client for quick API tests |
| SQLiteStudio | v3.4.x | GUI tool for database inspection and query verification |
| Windows Task Manager / htop | OS built-in | CPU and memory usage monitoring during load tests |

Table 6.3: Software Testing Environment

### 6.3.3 Testing Types Applied

| Testing Type | Description | Tools Used |
|---|---|---|
| Unit Testing | Individual API endpoints tested in isolation for correct logic, status codes, and database changes. | Postman, Thunder Client |
| Integration Testing | Testing interaction between the Node.js server, SQLite3 database, and frontend SPA to verify end-to-end data flow. | Browser + Postman |
| Functional Testing | Verifying that each feature delivers the expected output given valid and invalid inputs — covering all user roles. | Manual (Browser) |
| Boundary Testing | Testing input limits — e.g., empty fields, duplicate IDs, zero passengers, maximum stop index. | Postman |

| | | |
|---|---|---|
| Negative Testing | Deliberately supplying wrong credentials, invalid service numbers, and malformed JSON to verify error handling. | Postman |
| UI / Usability Testing | Manual walkthrough of all SPA views to verify layout, responsiveness, and data rendering on screen. | Chrome DevTools |
| Performance / Load Testing | Simulating concurrent API requests to /api/services to assess server response consistency. | Postman Runner |
| Regression Testing | Re-executing test cases after each code change to ensure existing functionality is not broken. | Manual |

Table 6.4: Testing Types Applied

## 6.4 DESIGN OF TEST CASES AND SCENARIOS

The test cases below are designed to systematically validate the key functional flows of the system. Each test case is identified by a unique Test Case ID, describes its objective, the input data, the expected output, and the actual result observed during validation.

### 6.4.1 Test Cases — Passenger Module

| TC ID | Test Scenario | Input | Expected Output | Actual Result | Status |
|---|---|---|---|---|---|
| TC-P01 | Valid bus search between two stops | From: Hyderabad, To: Vijayawada | Service 101 listed with stops, departure time 07:00, occupancy data | Service 101 displayed correctly | PASS |
| TC-P02 | Search with no matching route | From: Tirupati, To: Warangal | No results found message displayed | Empty result set returned, UI shows 'No buses found' | PASS |
| TC-P03 | Bus position at departure time | Service 101 queried at exactly 07:00 | Bus position shown at stop 0 (MGBS) | Stop index 0 rendered correctly | PASS |

| | | | | | |
|---|---|---|---|---|---|
| TC-P04 | Bus position midroute | Service 101 queried at 07:45 | Bus between Hayathnagar (45 min) and Choutuppal (90 min) | Progress bar correctly positioned between stops 2 and 3 | PASS |
| TC-P05 | Occupancy autorefresh | Conductor updates men_count to 30 | Passenger portal shows 30 men within 3 seconds | Refresh cycle updated display correctly | PASS |
| TC-P06 | Same origin and destination | From: Vijayawada, To: Vijayawada | No services listed or appropriate message | Empty result displayed | PASS |

Table 6.5: Test Cases — Passenger Module

### 6.4.2 Test Cases — Conductor Module

| TC ID | Test Scenario | Input | Expected Output | Actual Result | Status |
|---|---|---|---|---|---|
| TC-C01 | Valid conductor login | EID1001 / PIN: 1234 | HTTP 200, success: true, name: 'Ravi Kumar' | Login successful, dashboard loaded | PASS |
| TC-C02 | Invalid PIN login | EID1001 / PIN: 9999 | HTTP 401, success: false, 'Invalid Employee ID or PIN' | 401 returned with correct message | PASS |
| TC-C03 | Issue male ticket | type: men, service: 101, from: 0, to: 3 | fare: 120.00, fare_status: Paid, ticket ID generated | Record inserted correctly in tickets table | PASS |
| TC-C04 | Issue female ticket | type: women, service: 101, from: 0, to: 3 | fare: 0.00, fare_status: Free | Free ticket recorded correctly | PASS |

| | | | | | |
|---|---|---|---|---|---|
| TC-C05 | Advance stop index | PATCH /api/services/101/stop, current_stop_index: 3 | HTTP 200, 'Stop updated', DB updated | Stop index updated, reflected on passenger portal | PASS |
| TC-C06 | Update occupancy | PATCH /api/services/101/occupancy, men:25, women:10, students:5 | HTTP 200, occupancy updated in DB | Correct counts stored and displayed | PASS |
| TC-C07 | Correct PIN change | old_pin: 1234, new_pin: 5678 | HTTP 200, 'PIN updated successfully' | PIN changed in database | PASS |
| TC-C08 | Wrong old PIN change | old_pin: 0000, new_pin: 5678 | HTTP 401, 'Current PIN is incorrect.' | 401 returned | PASS |

Table 6.6: Test Cases — Conductor Module

### 6.4.3 Test Cases — Admin Module

| TC ID | Test Scenario | Input | Expected Output | Actual Result | Status |
|---|---|---|---|---|---|
| TC-A01 | Valid admin login | admin / admin123 | HTTP 200, success: true, name: 'Depot Manager' | Login successful | PASS |
| TC-A02 | Invalid admin login | admin / wrongpass | HTTP 401, success: false | 401 returned | PASS |
| TC-A03 | Admin stats dashboard | GET /api/admin/stats | JSON with totalServices: 41, activeConductors: 4, pendingAssignments, totalRevenue, totalPassengers | All stats returned correctly | PASS |
| TC-A04 | Add new conductor | EID: EID2001, Name: Test Conductor | HTTP 200, conductor added with status active | Record created in DB | PASS |
| TC-A05 | Duplicate conductor ID | EID: EID1001 (existing) | HTTP 409, 'Employee ID already exists' | 409 returned correctly | PASS |

| | | | | | |
|---|---|---|---|---|---|
| TC-A06 | Delete conductor | DELETE /api/conductors/EID2001 | HTTP 200, conductor deleted, assignments removed | Cascading delete worked | PASS |
| TC-A07 | Add new service with stops | Service no: 999, origin, destination, 3 stops | HTTP 200, service and stops inserted in transaction | Transaction completed successfully | PASS |
| TC-A08 | Duplicate service number | Service no: 101 (existing) | HTTP 409, 'Service Number already exists' | 409 returned | PASS |
| TC-A09 | Delete service cascade | DELETE /api/services/101 | Stops, tickets, assignments for service 101 deleted | All related records removed from DB | PASS |
| TC-A10 | Assign conductor to service | POST /api/assignments, EID1004, service 104 | HTTP 200, assignment created with status: pending | Assignment record created | PASS |

Table 6.7: Test Cases — Admin Module

### 6.4.4 Test Cases — Negative & Boundary Testing

| TC ID | Test Scenario | Input | Expected Output | Actual Result | Status |
|---|---|---|---|---|---|
| TC-N01 | Add service with missing fields | POST /api/services with no stops array | HTTP 400, 'Missing required fields or stops' | 400 returned with correct message | PASS |
| TC-N02 | Invalid assignment status | PATCH /api/assignments/1/status, status: 'maybe' | HTTP 400, 'Invalid status' | 400 returned | PASS |
| TC-N03 | Delete non-existent service | DELETE /api/services/XXXX | HTTP 404, 'Service not found' | 404 returned | PASS |
| TC-N04 | Delete non-existent conductor | DELETE /api/conductors/EIDXXXX | HTTP 404, 'Conductor not found' | 404 returned | PASS |
| TC-N05 | Zero passengers update | PATCH /api/services/101/occupancy, men:0, women:0, students:0 | HTTP 200, counts reset to 0 in DB | Occupancy reset correctly | PASS |

| | | | | | |
|---|---|---|---|---|---|
| TC-N06 | Empty ticket body | POST /api/tickets with empty JSON {} | HTTP 500 or implicit null handling | Server returns 500, logged for fix | FAIL* |

Table 6.8: Negative & Boundary Test Cases

TC-N06 Note: When an empty body is sent to POST /api/tickets, the server currently does not perform input validation before the INSERT query, resulting in a 500 error. A recommended fix is to add a check for required fields (service_no, type, from_stop_index, to_stop_index) at the start of the endpoint handler — mirroring the validation already implemented in the POST /api/services endpoint.

## 6.5 TEST RESULTS SUMMARY

| **Module** | **Total TCs** | **Pass** | **Fail** | **Pass Rate** |
|---|---|---|---|---|
| Passenger Module | 6 | 6 | 0 | 100% |
| Conductor Module | 8 | 8 | 0 | 100% |
| Admin Module | 10 | 10 | 0 | 100% |
| Negative & Boundary | 6 | 5 | 1 | 83.3% |
| TOTAL | 30 | 29 | 1 | 96.7% |

Table 6.9: Test Results Summary

Overall, 29 out of 30 test cases passed successfully, giving a pass rate of 96.7%. The single failure (TC-N06) is a minor input validation gap in the /api/tickets endpoint, which does not affect normal operation but should be addressed prior to production deployment. All core functional flows — passenger search, live tracking simulation, digital ticketing, conductor management, and admin operations — perform correctly and reliably under the test conditions described in this chapter.

# 7.CONCLUSION & FUTURE ENHANCEMENT

## CONCLUSION

The Smart Public Transport Management System was developed to simplify and improve the management of public bus transportation services. This project helps different users such as passengers, conductors, and administrators to perform their tasks in an easy and efficient way. Passengers can check bus services, routes, stop details, and ticket information. Conductors can manage ticket issuing, passenger occupancy, and service details, while administrators can handle service management, conductor assignments, and monitoring. By converting manual operations into a digital system, the project reduces paperwork, saves time, improves accuracy, and makes transport management more organized and user-friendly.

## Future Enhancement

In the future, this project can be improved by adding more advanced and real-time features to make the system even more useful. Features such as live bus tracking using GPS, online seat reservation, digital payment integration, and mobile application support can enhance passenger convenience. The system can also be upgraded with AIbased route optimization, real-time traffic updates, and automatic passenger analytics for better decision-making. Adding notification alerts, multilanguage support, and cloud database integration can further increase the scalability, accessibility, and overall performance of the Smart Public Transport Management System.

# 8.REFERENCES

The following references were used in the study, design, and implementation of the APSRTC Smart Public Transport Management System. References are arranged in the order of their appearance in the report. Web URLs were last accessed in April 2026.

## Books

**Research Papers (Literature Survey)**


1. Singh, R., and Kaur, G., "Real-Time Bus Tracking and Passenger Information System Using GPS and GSM," International Journal of Computer Applications, vol. 175, no. 12, pp. 1-7, 2020.

2. Gupta, A., Verma, R., and Sharma, P., "Digital Ticketing System for Public Transport: A Web-Based Approach," International Journal of Innovative Research in Science and Technology, vol. 8, no. 3, pp. 45-52, 2021.

3. Lavanya, K., and Rama Rao, T., "IoT-Based Smart Bus Management System for Andhra Pradesh," Journal of Emerging Technologies and Innovative Research (JETIR), vol. 6, no. 5, pp. 312-319, 2019.

4. Al-Smadi, M., Shaalan, K., and Al-Radaideh, Q., "A Review of Public Transport Passenger Information Systems," IEEE Transactions on Intelligent Transportation Systems, vol. 21, no. 9, pp. 3847-3862, 2020.

5. Vijayalakshmi, S., and Rajesh Kumar, B., "Role-Based Access Control in Public Transport Management Portals," International Journal of Advanced Research in Computer Science, vol. 13, no. 2, pp. 98-107, 2022.

6. Mehta, R., and Patel, D., "Node.js and SQLite for Lightweight Government Web Applications: A Case Study," Journal of Web Engineering, vol. 20, no. 4, pp. 1123–1145, 2021.

7. Nair, P., and Suresh Babu, “Free Travel Scheme Tracking for State Transport Undertakings: Digital Approaches,” Indian Journal of Transport Management, vol. 47, no. 1, pp. 23-38, 2023.

8. Bhatt, A., Joshi, N., and Kulkarni, S., “RESTful API Design Patterns for Transport Management Systems,” International Journal of Software Engineering and Applications (IJSEA), vol. 13, no. 6, pp. 11-28, 2022.

9. Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts, 7th ed., McGraw-Hill Education, New York, 2019, pp. 1-1376.

10. Sommerville, I., Software Engineering, 10th ed., Pearson Education, London, 2016, pp. 1-810.

## Web References

11. Express.js Foundation, “Express - Fast, Unopinionated, Minimalist Web Framework for Node.js,” Official Documentation, 2024. Available: https://expressjs.com/en/4x/api.html

12. SQLite Consortium, “SQLite Documentation - SQL As Understood by SQLite,” SQLite.org, 2024. Available: https://www.sqlite.org/lang.html

13. Andhra Pradesh State Road Transport Corporation (APSRTC), “Official Website - Bus Schedules, Fares and Services,” Government of Andhra Pradesh, 2024. Available: https://www.apsrtc.ap.gov.in

14. MDN Web Docs, “Fetch API - Web APIs,” Mozilla Developer Network, 2024. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

15. Node.js Foundation, “Node.js Official Documentation,” 2024. Available: https://nodejs.org/en/docs

16. Ministry of Urban Development, Government of India, “National Urban Transport Policy,” 2014. Available: https://mohua.gov.in